SOURCE-LEVEL GUIDE · SEARXNG/SEARXNG

SearXNG：从原理到源码

一份给想读懂、贡献或定制 SearXNG 的工程师的导读：一个不追踪用户的元搜索引擎，怎样把一次查询同时发给几十个搜索引擎、在 3 秒内收回结果、去重打分，再保护好用户和实例本身。

searxng/searxng @ 3a3dd15

怎么读这份导读

这份导读给三类读者：想**读懂** SearXNG 怎么工作的人，想给它**提 PR** 的人，以及想**部署或定制**一个自己的实例（比如给本地大模型当搜索工具）的人。

全文按一次查询走过的路径展开：

1. **全景**：元搜索要解决什么问题，SearXNG 的目标和限制。
2. **入口与配置**：进程怎么启动，`settings.yml` 怎么叠加。
3. **请求预处理**：偏好（cookie）、查询语法（`!gh`、`:zh`、`<3`）、组装 `SearchQuery`。
4. **搜索调度**：每个引擎一个线程、统一的截止时间、失败引擎的挂起退避。
5. **引擎**：一个引擎就是一个 Python 模块，`request()` + `response()`。
6. **网络层**：`curl_cffi` 冒充 Chrome、每个引擎独立的连接池和代理。
7. **结果合并与排序**：去重规则和评分公式。
8. **插件、答题器与输出**：三个钩子、四种输出格式。
9. **隐私与防滥用**：不追踪用户的那几处设计，以及 limiter 怎么挡机器人。
10. **端到端**：把 `!gh vllm` 从按下回车到看到结果串一遍。
11. **部署形态、许可证（AGPL）、贡献与定制、源码索引**。

正文里的源码链接都指向 GitHub 上固定 commit 的代码行，上游继续改动也不会漂移。HTML 版里把鼠标悬停在图的节点上，会高亮它的连线。

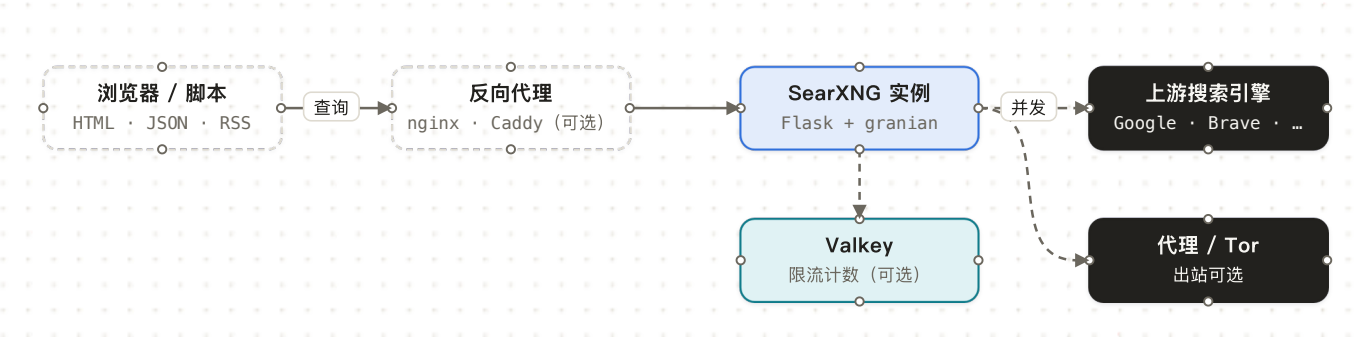
> **要点 · 固定版本**
>
> `searxng/searxng` @ `3a3dd15`（2026-11-08，master）。Python 约 5.2 万行，`searx/engines/` 下 255 个引擎模块，默认配置里登记了 353 个引擎条目。

全景：元搜索要解决什么问题

没有它时的问题

环节	问题	SearXNG 的做法
隐私	直接用 Google、Bing 时，搜索词、IP、cookie、点击都被记录并用于画像	由实例代你去问：上游只看到实例的 IP；偏好只存在你浏览器的 cookie 里；结果链接不带 Referer
覆盖面	单个搜索引擎的索引和排序有偏向；论文、代码、包、图片、地图分散在不同的站点	同时查询多个引擎（通用、图片、视频、新闻、学术、IT、地图……），合并去重、统一打分
可控性	无法自己决定用哪些来源、屏蔽哪些站点、改写链接	一个 YAML 文件开关引擎、调权重；插件可以删除、提权或改写结果
可编程	搜索 API 多数收费、要申请 key	自建实例后可以开 `json` 格式，给脚本和 LLM 当免费的搜索 API

全景：一次查询经过的地方



实例本身没有索引，所有结果都来自上游；它的工作是转发、合并和保护隐私。入口见 `webapp.search()`，出站在 `network/client.py`。

类比 · 帮你比价的代购

你把要买的东西告诉代购（搜索词），代购同时去好几家店问价（各个引擎），店家只看到代购，不知道是你在买（隐私）。几家店卖的同一件商品只算一件，哪家都有的排前面（去重和打分）。代购只给每家店 3 秒（超时），哪家店连着出问题就先别去了（挂起）。

要求、目标与限制

后面每一章都在回答这张表里的某一行。

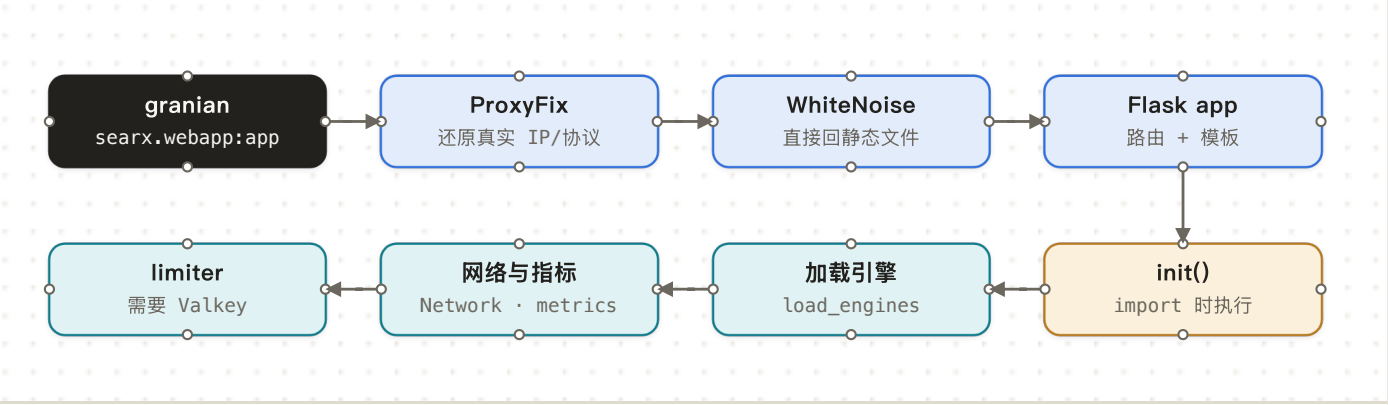
项目	内容	在哪一章解决
核心目标	不追踪、不画像用户的元搜索引擎	隐私与防滥用
功能要求	多引擎并发、合并去重、分类标签、多语言、多种输出格式、可扩展的引擎和插件	引擎；结果合并；插件与输出
延迟限制	一个页面要等所有引擎；默认每个引擎只给 3 秒，最慢的不能拖住整页	搜索调度
外部限制	上游会限流、出验证码、封 IP，接口和页面结构随时变；有的只认真实浏览器的 TLS 指纹	网络层；搜索调度（挂起）
滥用风险	公开实例会被机器人当免费搜索 API 刷，进而被上游封锁	隐私与防滥用（limiter）
部署形态	个人、家庭、公开实例都要能跑；配置要能只写差异	部署形态
许可证	AGPL-3.0-or-later：对外提供服务时要提供修改后的源码	许可证

入口与配置

进程怎么起来

`searx/webapp.py` 在**被 import 的时候**就完成全部初始化：模块末尾直接调用 `init()`，所以无论用 granian、uwsgi 还是 `python -m searx.webapp`，加载这个模块就会加载所有引擎、建好网络和插件。

启动顺序与 WSGI 包装



外层包装见 `webapp.py 末尾`；`init()` 依次初始化缓存库、favicon、语言、Valkey、插件、`search.initialize`（引擎、网络、指标、processor），最后装上 `limiter`。容器里用 `granian` 启动。

两个启动期检查值得记住：生产模式下 `server.secret_key` 还是默认的 `ultrasecretkey` 就直接退出（`init`）；`public_instance: true` 但连不上 Valkey 也会退出（`limiter.initialize`）。

配置怎么叠加

默认配置是仓库里的 `searx/settings.yml`（约 3500 行，大部分是引擎列表）。你自己的配置通常只写差异：

```yaml
use_default_settings: true      # 先读默认配置，再叠加下面的值
server:
  secret_key: "…"
  limiter: false
search:
  formats: [html, json]         # 默认只有 html；脚本要用 JSON 必须打开
```

机制	作用	代码
配置文件位置	`SEARXNG_SETTINGS_PATH`（文件或目录）→ `/etc/searxng`	`get_user_cfg_folder`
`use_default_settings`	按键合并到默认配置；`engines.remove` / `keep_only` 删减引擎	`update_settings`
环境变量覆盖	`SEARXNG_SECRET`、`SEARXNG_PORT` 等直接覆盖对应键	`SettingsValue`
类型与默认值	每个键的类型、默认值、合法取值	`settings_defaults.py`

注意 · 容易误解

`search.formats` 默认只有 `html`（`settings.yml`）。用 `format=json` 请求一个没打开 JSON 的实例会得到 403，而不是空结果，见 `search()` 开头的检查。

请求预处理：偏好、查询语法与 SearchQuery

偏好从哪来

SearXNG 没有账号，也没有服务端会话。每个请求进来时，`pre_request()` 新建一个 `Preferences`，依次用这些来源填它：

1. 浏览器的 **cookie**（你在偏好页保存的设置）；
2. 本次请求的 GET/POST 参数，或一个编码过的 `preferences` 参数（可以把整套设置写进一个 URL 带走）；
3. 都没有时，用浏览器的 `Accept-Language` 推断搜索语言和界面语言。

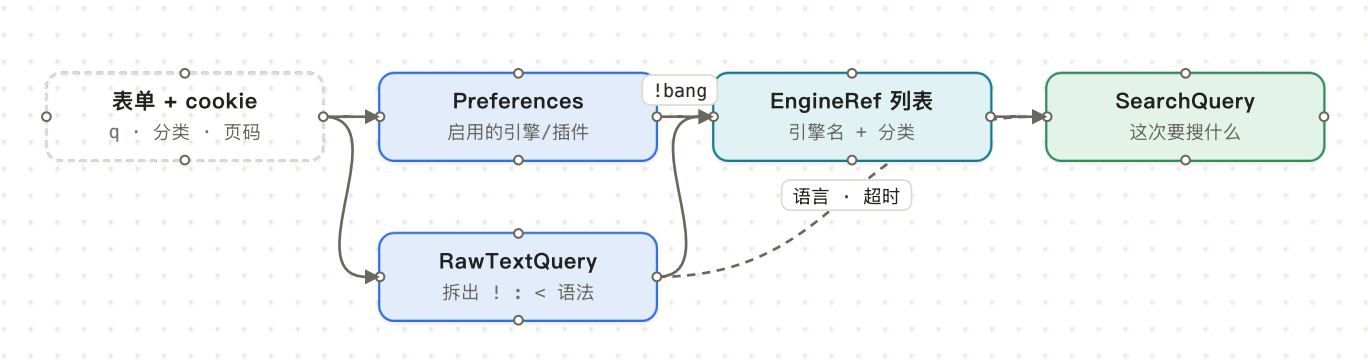
它还算出这个请求启用了哪些插件，存在 `sxng_request.user_plugins`。整个过程不落盘，这是“不追踪”在代码层面的落点之一：服务端不需要也不保存任何用户身份。

查询语法

搜索框里的文字先交给 `RawTextQuery`：按空白切开，每一段依次问几个解析器“这是不是你的语法”。

写法	解析器	作用
`<3`	`TimeoutParser`	本次搜索的超时上限（秒）
`:zh`、`:en-US`	`LanguageParser`	搜索语言 / 地区
`!gh`、`!images`	`BangParser`	只用某个引擎（按 shortcut 或名字）或某个分类
`!!w` 量子	`ExternalBangParser`	外部 bang：直接 302 跳到别的网站搜
`!!` 单独出现	`FeelingLuckyParser`	直接跳到第一条结果

从表单到 SearchQuery



组装在 `get_search_query_from_webapp`：有 `!bang` 时用查询里指定的引擎，否则按所选分类和偏好展开；再去重、过滤掉用户禁用或需要 token 的引擎。结构定义见 `SearchQuery` 和 `EngineRef`。

`SearchQuery` 是后面所有组件的输入：查询词（去掉了语法部分）、`EngineRef` 列表、语言、页码、时间范围、safesearch、超时上限，以及 `engine_data`（个别引擎需要跨页保存的数据，比如翻页 token，放在 HTML 表单里往返，服务端仍然不保存状态）。

搜索调度：线程、截止时间与挂起

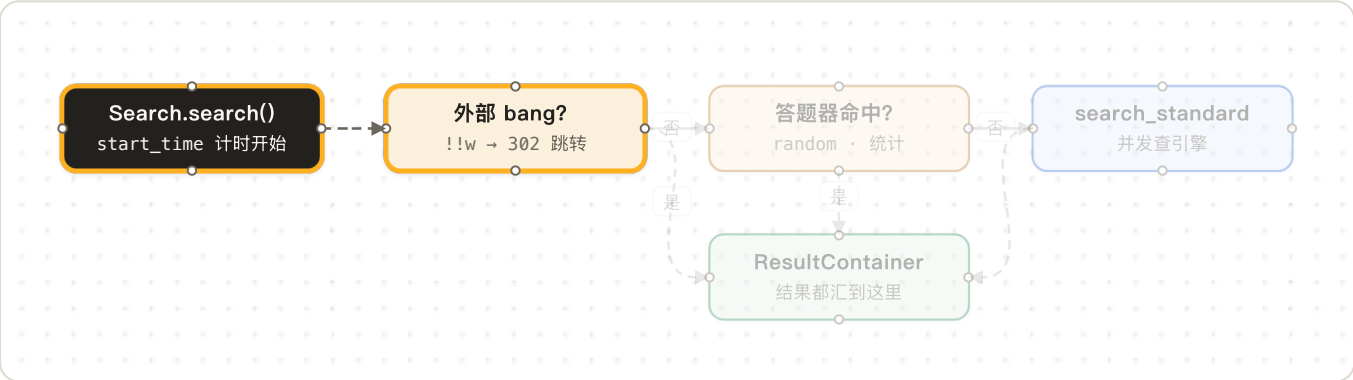
职责与难点

调度要在一个固定的时间预算里，把同一个查询发给可能几十个引擎，并且**最慢的那个不能拖住整页**。上游还会时好时坏：限流、验证码、超时，失败的引擎要先别去打扰它。

三条路径

`Search.search()` 先尝试两条捷径，都不满足才真正去查引擎：

▶ 一次搜索走哪条路 1 / 3



1. 先看外部 bang

查询里有 `!!w` 这类外部 bang 时，`search_external_bang` 只算出跳转地址，整个搜索到此结束，webapp 返回 302。

2. 再问答题器

答题器是不联网的小功能，比如“random uuid”“avg 1 2 3”。`search_answerers` 有结果就直接返回，不再查任何引擎。

3. 否则并发查引擎

`search_standard` 为每个选中的引擎准备请求，再为每个引擎起一个线程，结果都写进同一个 `ResultContainer`。

每个引擎一个线程，统一的截止时间

`_get_requests` 对每个 `EngineRef`：

1. 找到它的 processor；**引擎处于挂起状态就跳过**，并把它记为“无响应（已挂起）”；

2. 调 `processor.get_params()` 生成请求参数；引擎不支持当前页码或时间范围时返回 `None`，同样跳过；
3. 记下所有引擎超时的最大值，作为默认超时。

实际超时再和两个上限比较：用户在查询里写的 `<N`，以及管理员配置的 `outgoing.max_request_timeout`：

`max_request_timeout`	用户写了 `<N`	实际超时
未设置	没写	所有引擎超时的最大值（默认 3 秒）
未设置	写了	min(默认值, N)
设置了	没写	min(默认值, max)
设置了	写了	min(N, max)

然后 `search_multiple_requests` 给每个引擎起一个线程，线程名都是同一个 `search_id`。等待时用的是**剩余预算**：`th.join(actual_timeout - 已用时间)`。到点还没回来的线程不会被杀掉，只被标记为超时，引擎记为“timeout”，页面不再等它，它晚到的结果也会被丢掉（`ResultContainer` 关闭后忽略新结果，见 `extend`）。

> **类比 · 统一收卷的考场**
>
> 所有考生（引擎）同时开考，监考老师只看墙上一个钟（`start_time` + `actual_timeout`）。到点收卷，没交的记“缺考”，不会因为某个人写得慢而推迟收卷。连续作弊被抓（验证码、429）的考生下几场禁考（挂起），禁考时间越来越长，但有上限。

挂起与退避

失败分两类：普通异常只记录错误；网络超时、SSL 错误、请求异常、验证码、429、403 这几类会**挂起**引擎（`OnlineProcessor.search` 里的 `suspend=True`）。挂起时长由 `SuspendedStatus.suspend` 决定：异常自带时长就用它（比如上游返回了 Retry-After），否则用 `search.ban_time_on_fail`（默认 5 秒），上限 `search.max_ban_time_on_fail`（默认 120 秒）。之后有一次成功，`extend_container` 就调用 `resume()` 解除。

> **注意 · 容易误解**
>
> 挂起状态是按引擎存在**进程内存**里的（`SUSPENDED_STATUS`）。用 granian 开多个 worker 进程时，每个进程各自记一份；一个 worker 里挂起的引擎，另一个 worker 可能还在请求。

引擎：一个模块，两个函数

契约

一个引擎就是 `searx/engines/` 下的一个 Python 模块。最小的契约只有两个函数：

名字	作用	说明
`request(query, params)`	填 `params["url"]`，需要时改 `method`、`headers`、`data`、`cookies`	`params` 里已有 `pageno`、`safesearch`、`time_range`、`searxng_locale` 等，见 `RequestParams` 和 `OnlineParams`
`response(resp)`	解析 HTTP 响应，返回结果列表	每条结果至少有 `url`、`title`、`content`；可以指定 `template`（图片、视频、包、论文……）
模块属性	`categories`、`paging`、`time_range_support`、`about` 等	未写的用 `ENGINE_DEFAULT_ARGS` 的默认值
`setup(engine_settings)`、`init()`	可选：加载时检查配置、启动时预热	`call_engine_setup`

GitHub 引擎是很好的最小例子，整个文件 65 行：

```
# 简化自 searx/engines/github.py
categories = ['it', 'repos']
search_url = 'https://api.github.com/search/repositories?sort=stars&order=desc&{query}'

def request(query, params):
    params['url'] = search_url.format(query=urlencode({'q': query}))
    params['headers']['Accept'] = accept_header
    return params

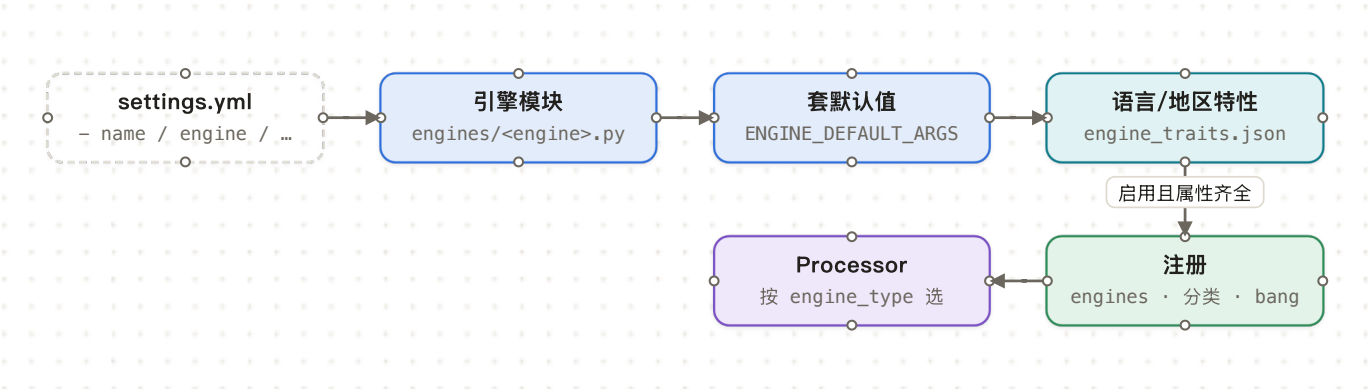
def response(resp):
    return [{'template': 'packages.html', 'url': item['html_url'],
             'title': item['full_name'], 'popularity': item['stargazers_count'], ...}
            for item in resp.json().get('items', [])]
```

见 `github.request` 和 `github.response`。

一个模块可以变成多个引擎

`settings.yml` 里的每一项 `- name: … engine: …` 都是一个**引擎实例**。`load_engine` 按 `engine:` 加载模块，套上默认值，再用这一项的配置覆盖。所以同一个模块可以配置出多个引擎：通用的 `xpath` 和 `json_engine` 模块，只靠配置里的 URL 和 XPath / JSON 路径，就能接入一个新网站，不用写代码。

从配置项到可用的引擎



加载流程见 `load_engine`：名字不能有下划线、`inactive` 的跳过、缺必填属性的跳过、`setup()` 失败的跳过。语言和地区映射来自离线生成的 `engine_traits.json`（`EngineTraits`）。processor 的选择在 `ProcessorMap.init`。

几种 processor

`engine_type` 决定由哪个 processor 来跑这个引擎（`searx/search/processors/`）：

engine_type	做什么
`online`	绝大多数引擎：发 HTTP 请求，解析响应
`offline`	不联网，比如查本地数据库或命令
`online_dictionary`、`online_currency`、`online_url_search`	先判断查询是否像“翻译”“汇率”“URL”，不像就不发请求

语言和地区：traits

每个上游对语言、地区的写法都不一样（`zh-CN`、`zh-hans`、`cn`……）。SearXNG 用离线脚本抓各引擎支持的语言列表，生成 `engine_traits.json`；运行时引擎调用 `traits.get_language` / `get_region` 把用户的 `searxng_locale` 翻译成自己的写法。更新脚本在 `searxng_extra/update/`。

注意 · 容易误解

引擎坏掉最常见的原因是上游改了页面或接口，不是 SearXNG 的 bug。修引擎的 PR 基本都是改 `response()` 里的 XPath / JSON 路径，或者 `request()` 里的参数，比如这个 commit 的 `seznam` 加了 `noredirect=1`。

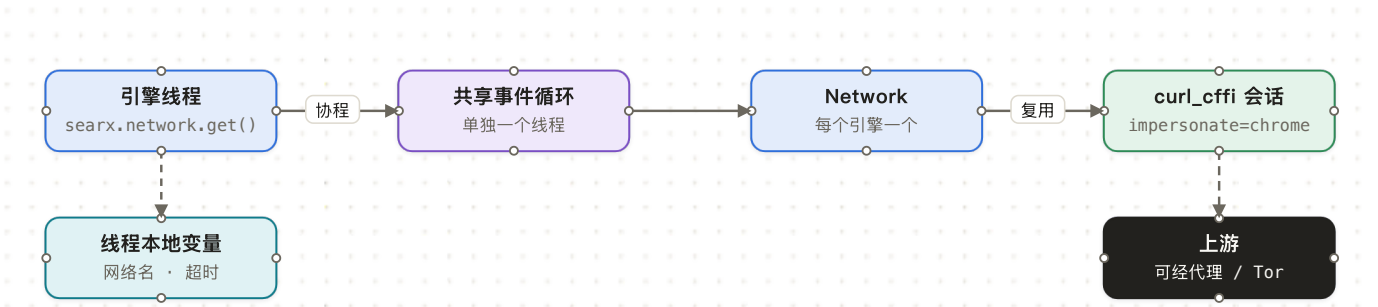
网络层：冒充浏览器的连接池

难点

很多上游会根据 TLS 和 HTTP/2 的握手特征（俗称 TLS 指纹）识别非浏览器客户端，直接返回验证码或拒绝。另外每个引擎可能要走不同的代理、出口 IP、HTTP 版本，而几十个线程同时发请求，又不能每个都开一套事件循环。

原理

引擎线程怎么发出 HTTP 请求



同步接口 `network.request` 把协程交给共享的事件循环（`client.init` 起的 `asyncio_loop` 线程），再用剩余超时等结果。`Network` 按引擎名建立（`network.initialize`），客户端在 `new_client` 里创建，默认 `impersonate="chrome"`。

关键点	说明	代码
冒充 Chrome	用 `curl_cffi` 发请求，默认模仿 Chrome 的 TLS / HTTP2 指纹；可设 `impersonate: none` 关掉	`DEFAULT_IMPERSONATE`
每个引擎一个 Network	引擎可以在配置里写 `network:`，设置代理、出口地址、重试、HTTP/2、连接数	`Network`
代理和出口 IP 轮换	多个代理或本地地址时按轮询使用	`get_proxy_cycles`
Tor	代理都是 `socks5h://` 时，先确认确实走了 Tor	`check_tor_proxy`
超时随线程走	processor 把截止时间写进线程本地变量，每个请求按剩余时间等待	`set_timeout_for_thread`

历史

网络层在 2026-09-03 从 httpx 迁到了 `curl_cffi`（commit `be836e61` “[mod] network: migrate to curl_cffi”，经 GitHub API 核实）。网上旧文档和旧 issue 里提到的 httpx、`HTTPX` 配置项，在这个版本里已经不适用。

结果合并与排序

结果汇到哪里

所有引擎线程都往同一个 `ResultContainer` 写。`extend()` 把每条结果按类型分流：普通结果、答案（answer）、建议（suggestion）、纠错（correction）、信息框（infobox）、以及给下一页用的 `engine_data`。每条结果写入前都要先过插件的 `on_result` 钩子，插件返回 `False` 就丢掉这条。

什么算同一条结果

普通结果用哈希判重（`MainResult.hash`）：**模板相同、URL 去掉协议后相同（http 和 https 算一条）、图片地址相同。**判为重复时，`merge_two_main_results` 把两条合并（保留更长的内容等），并把这次出现的名次追加到 `positions`。

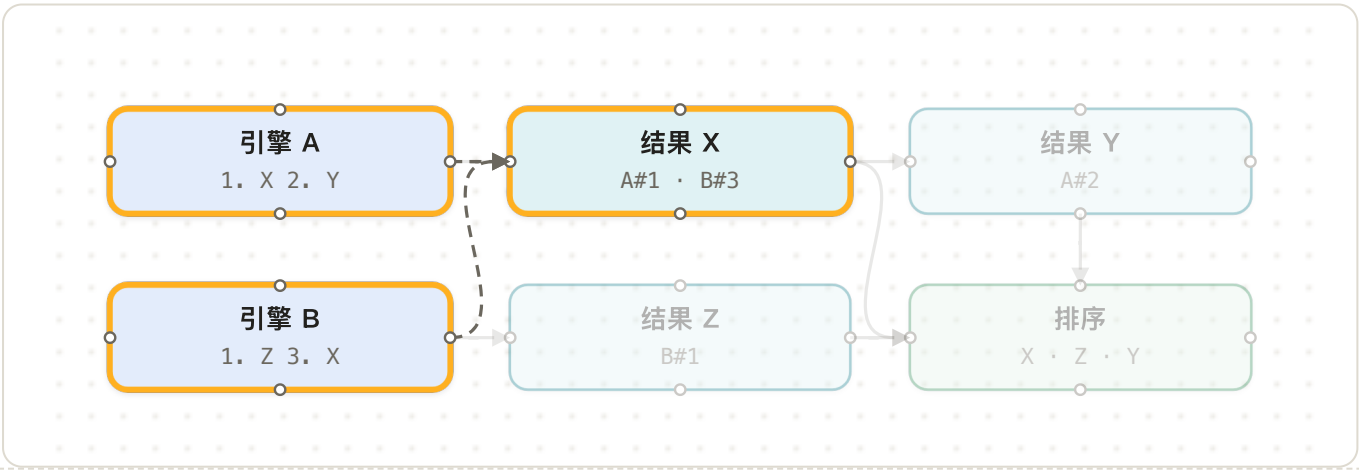
怎么打分

打分在 `calculate_score`，规则很短：

- `weight` = 所有返回过这条结果的引擎的权重相乘（默认 1.0），再乘以出现的次数；
- `score` = 对每次出现的名次求和 `weight / position`；
- 插件可以把结果标为 `high` 优先级（每次出现加满 `weight`，不除名次）或 `low`（得 0 分）。

也就是说：**被越多引擎返回、名次越靠前，分越高**，而且“多个引擎都返回”的效果是乘法的。

▶ **三条结果怎么排序（数字由公式计算得出）** 1 / 4



1. 合并

X 被 A（第 1 名）和 B（第 3 名）都返回，URL 相同，合并成一条：`positions = [1, 3]`，`engines = {A, B}`。

2. 算 X 的分

`weight = 1.0 × 1.0 × 2 = 2.0`；`score = 2/1 + 2/3 ≈ 2.667`。

3. 算 Z 和 Y

Z 只出现一次，第 1 名：`weight = 1`，`score = 1/1 = 1.0`。Y 只出现一次，第 2 名：`score = 1/2 = 0.5`。

4. 排序

按分数降序：X（2.667）、Z（1.0）、Y（0.5）。如果把引擎 B 的 `weight` 设成 2，X 的分会变成 `4/1 + 4/3 ≈ 5.333`。

第二轮：按类型聚拢

只按分数排，图片、视频、普通网页会交错出现。`get_ordered_results` 再做一轮：同一分类、同一模板（有无缩略图也算区别）的结果，在 20 个位置以内会被往前聚到一起，每组最多再吸收 8 条。

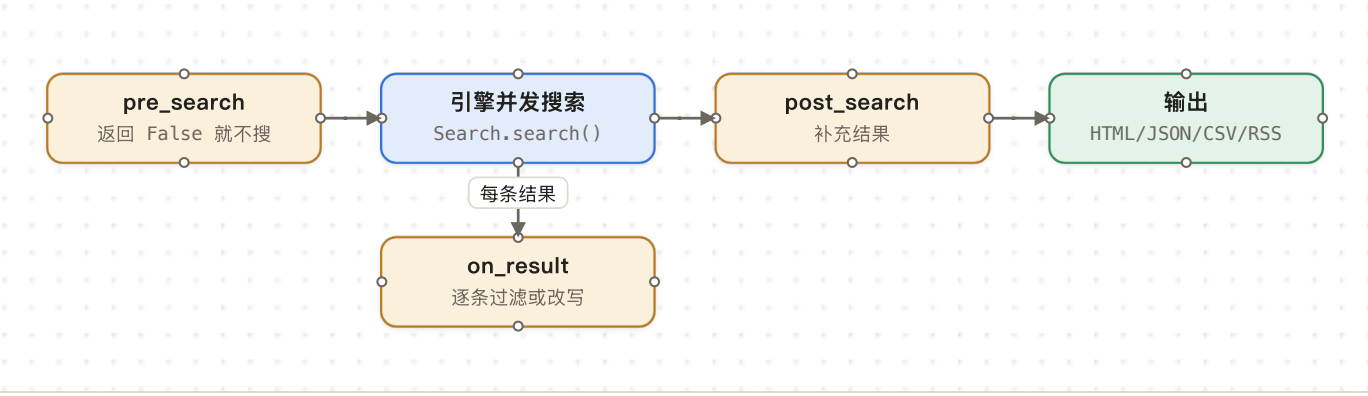
> **注意 · 容易误解**
>
> `weight` 是在**结果层面**按引擎相乘的，不是“引擎可信度”的加权平均。给一个引擎设很大的权重，会让它独有的结果和它参与的重复结果都大幅靠前，调之前最好先在测试实例里看效果。

插件、答题器与输出

插件的三个钩子

插件是一个继承 `Plugin` 的类，在 `settings.yml` 的 `plugins:` 下按类路径开关（默认列表）。用户也可以在偏好页逐个开关。

插件挂在搜索的哪里



调度在 `SearchWithPlugins.search`；三个钩子的分发在 `pre_search`、`on_result`、`post_search`。任何一个插件返回 `False`，后面的插件就不再执行。

几个内置插件正好展示了三种钩子的用法：

插件	钩子	作用
`hostnames`	`on_result`	按正则改写域名（比如 YouTube → Invidious）、删除某些站点、提高某些站点的优先级
`tracker_url_remover`	`on_result`	去掉链接里的跟踪参数
`calculator`、`unit_converter`	`post_search`	查询像算式或单位换算时补一个答案
`self_info`	`post_search`	搜“ip”“user agent”时显示你自己的信息

答题器

答题器（`AnswerStorage.ask`）比插件更早：按查询的第一个关键词匹配，命中就直接给答案，**完全不查引擎**。例子是 `random`（随机数、UUID）和 `statistics`（平均值、求和）。

四种输出

`OUTPUT_FORMATS` 是 `html`、`csv`、`json`、`rss`，每个实例在 `search.formats` 里决定开放哪几种。JSON 由 `get_json_response` 生成，CSV 由 `write_csv_response` 生成；HTML 用 Jinja 模板 `results.html` 渲染，每种结果模板在 `result_templates/`。前端主题的源码在 `client/simple`（TypeScript + Less，Vite 构建），产物提交在 `searx/static/themes/simple/`。

隐私与防滥用

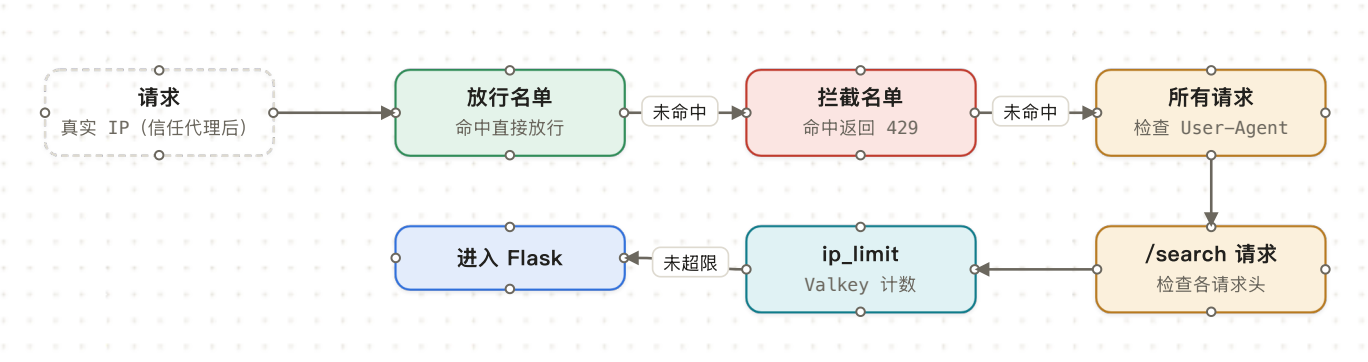
“不追踪”落在哪几处代码

措施	做法	代码
不保存用户	没有账号和服务端会话；偏好在 cookie 或 URL 里	`pre_request`
上游看不到你	请求由实例发出；上游只看到实例的 IP 和冒充的浏览器指纹	`_send_http_request`
不泄露来源	全站 `Referrer-Policy: no-referrer`，结果链接带 `rel="noreferrer"`	`settings.yml`、`macros.html`
图片代理	开启后缩略图经实例转发；URL 带 HMAC 签名，防止被当成任意代理	`image_proxify`、`image_proxy`
去跟踪参数	结果链接里的 utm 等跟踪参数被删掉	`tracker_url_remover`

limiter：挡住把实例当免费 API 的机器人

公开实例的大敌是脚本把它当免费搜索 API 刷，结果是实例的 IP 被上游封掉，所有人都用不了。limiter 是一个 `before_request` 钩子，规则在 `filter_request`：

limiter 的判断顺序



按顺序逐项检查，任何一项返回响应就直接拦下。请求头检查在 `botdetection/` 下各模块；计数在 `ip_limit`：20 秒内超过 15 次、或 10 分钟内超过 150 次就拦（可疑 IP 的阈值更低）。配置在 `limiter.toml`。

几个关键点：

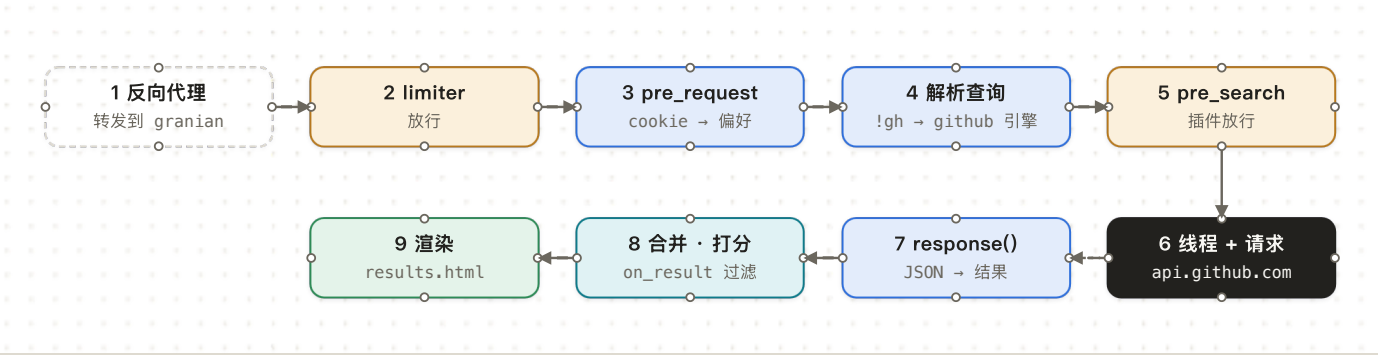
- **limiter 依赖 Valkey**（Redis 的开源分支）来跨进程计数。没有 Valkey 时 `limiter: true` 不会生效；`public_instance: true` 时直接拒绝启动（`limiter.initialize`）。
- **link_token**：公开实例会自动打开。页面里嵌一个 CSS 链接，真实浏览器会去请求它，脚本通常不会；没请求过的 IP 被视为可疑，适用更低的阈值（`is_suspicious`）。

- **信任的反向代理**：只有来自 `trusted_proxies` 的请求，才会从 `X-Forwarded-For` 取真实 IP。实例放在 nginx 后面却没配置，所有用户会被当成同一个 IP，一起被限流。

端到端：一次 `!gh vllm` 的完整路径

把前面的组件串起来。用户在搜索框输入 `!gh vllm` 并回车：

一次查询的 9 步



全部在这一个仓库里完成，没有闭源部分；唯一的外部系统是上游（这里是 GitHub 的搜索 API）。耗时主要在第 6 步，页面响应头里的 `Server-Timing` 会列出每个引擎的耗时（`post_request`）。

1. 反向代理把请求交给 granian，`ProxyFix` 还原真实 IP。
2. limiter（如果开了）检查名单、请求头和 IP 计数。
3. `pre_request` 从 cookie 和参数组出 `Preferences`，算出启用的插件。
4. `search()` 调 `get_search_query_from_webapp`：`!gh` 被 `BangParser` 识别为 GitHub 引擎的 shortcut（`settings.yml`），查询词剩下 `vllm`。
5. `SearchWithPlugins.search` 先跑插件的 `pre_search`。
6. 不是外部 bang，答题器没命中，于是 `search_standard`：为 github 引擎起一个线程，`github.request()` 填好 URL，网络层用冒充 Chrome 的 `curl_cffi` 会话发出请求。
7. `github.response()` 把 JSON 转成 `packages.html` 模板的结果。
8. 每条结果过插件的 `on_result`，进入 `ResultContainer` 合并、打分；`post_search` 补充答案。
9. 按 `format` 输出：这里是 HTML，用 `results.html` 渲染。

如果是不带 bang 的普通查询，第 6 步会同时起十几个线程（默认 general 分类下启用的引擎），第 8 步的去重和打分才真正发挥作用。

部署形态与配置

形态	怎么跑	要注意
容器（官方推荐之一）	镜像 `searxng/searxng`，入口脚本用 granian 启动 `Dockerfile`	配置目录挂到 `/etc/searxng`；首次启动会生成 `settings.yml`
脚本安装	`utils/searxng.sh` 装到系统，配 uwsgi 或 granian + nginx	适合长期运行的公开实例
开发	`make install` 建 venv，`make run` 启动开发服务器	见下一章

个人或小团队实例的最小配置：`use_default_settings: true`，改 `secret_key`，`limiter: false`，监听 `127.0.0.1`，需要给脚本用就在 `search.formats` 里加 `json`。

> **一个真实例子：给本地大模型当搜索工具**
>
> LatteCrew 在 RTX 5090 机器上就是这样用的：容器只映射到 `127.0.0.1:8888`，`formats: [html, json]`，`limiter: false`，`safe_search: 1`。聊天机器人的 `web_search` 工具请求 `/search?q=…&format=json`，从 `results` 里取标题、链接和 `content` 摘要。实测一次查询返回 37 条结果（主要来自 Brave 和 Google）；DuckDuckGo 当时返回验证码，被记为无响应引擎（JSON 里的 `unresponsive_engines`），其他引擎不受影响，这正是上面“挂起”机制的效果。

扩展规模时：开多个 granian worker 需要注意，挂起状态和部分缓存是进程内的（见搜索调度一章）；跨进程共享的只有 Valkey 里的限流计数，以及 SQLite 文件里的缓存（`ExpireCacheSQLite`）。

开源许可证

> **注意 · 不是法律意见**
>
> 以下是对许可证文本的通俗说明，涉及商业部署请咨询律师。

部分	许可证	证据
全部 Python 代码、模板、主题	AGPL-3.0-or-later	`LICENSE`；每个源文件头部的 `SPDX-License-Identifier`；`README`
Python 依赖	各自的许可证（Flask BSD、curl_cffi MIT 等）	`requirements.txt`
前端 npm 依赖	各自的许可证	`package.json`

AGPL 的核心：和 GPL 一样，分发修改后的程序时要提供源码；多出来的一条是**网络条款**（第 13 条）：如果你修改了程序，并通过网络让别人使用它，就要让这些用户能拿到你修改后的源码。

场景	可以吗	要做什么
自己或家里用，不改代码	可以	不需要做什么
改了代码，只自己用	可以	不需要公开
给上游提 PR	可以	代码以 AGPL 贡献；遵守 AI 政策
公开 fork	可以	保持 AGPL，保留版权和许可证声明
改了代码，开放给别人用（包括团队内网服务）	可以	要让使用者能获取你修改后的完整源码（页面上通常放一个源码链接，SearXNG 的界面本身就有这个位置：`brand.public_instances` 旁的 `git_url` 设置）
把它嵌进闭源产品一起分发	通常不行	整个衍生作品需要以 AGPL 开源
只通过 HTTP 调用一个实例的 JSON 接口	可以	你的程序没有包含 SearXNG 的代码，一般不受 AGPL 约束

历史：许可证一直是 AGPL v3 或更高版本。2013 年 searx 时期的 `LICENSE` 文件只有一段声明（commit `554f3a95`），2015 年换成完整的 AGPLv3 文本（commit `a905e4e7`，“full AGPLv3+ license according to #382”）。这两条都经 GitHub API 核实。SearXNG 仓库创建于 2021-04-12，保留了 searx 的完整提交历史。

例外：仓库里没有单独的商标、图标或 CLA 条款（在 README、CONTRIBUTING 和 LICENSE 中没有找到）；“SearXNG”名字和 logo 的使用没有书面说明，公开 fork 改名是更稳妥的做法。

贡献与定制

开发环境

照抄自 `CONTRIBUTING.rst` 和 `Makefile`：

```
git clone https://github.com/searxng/searxng.git && cd searxng
make install      # 建 Python venv，装依赖（./manage pyenv.install）
make run          # 启动开发服务器（./manage webapp.run）
make test         # yamllint、black、pyright、pylint、单元测试、robot 测试……
make format       # 格式化 Python 和 shell
make themes       # 构建前端主题（需要 Node）
```

也可以直接用 Flask 的开发服务器：`flask --app searx.webapp run --debug --reload --host 127.0.0.1 --port 8888`（`run()` 的文档里给出）。

项目的规矩

- **PR 模板**：说明做了什么、怎么在本地测试、关联的 issue，并勾选遵守 AI 政策（`PULL_REQUEST_TEMPLATE.md`）。
- **AI 政策**（`AI_POLICY.rst`）很严格：使用 AI 必须声明工具和程度；提交者必须完全理解代码；AI 不能是 PR 的主要作者；issue 和 PR 描述必须由人写；明显的 AI 生成垃圾会被永久拉黑。
- **大改动先开 issue 讨论**；commit 信息用现在时、祈使语气，并带 `[fix]`、`[mod]`、`[upd]` 这类前缀（从提交历史可以看出）。
- 翻译走 Weblate，不要直接改 `translations/`。

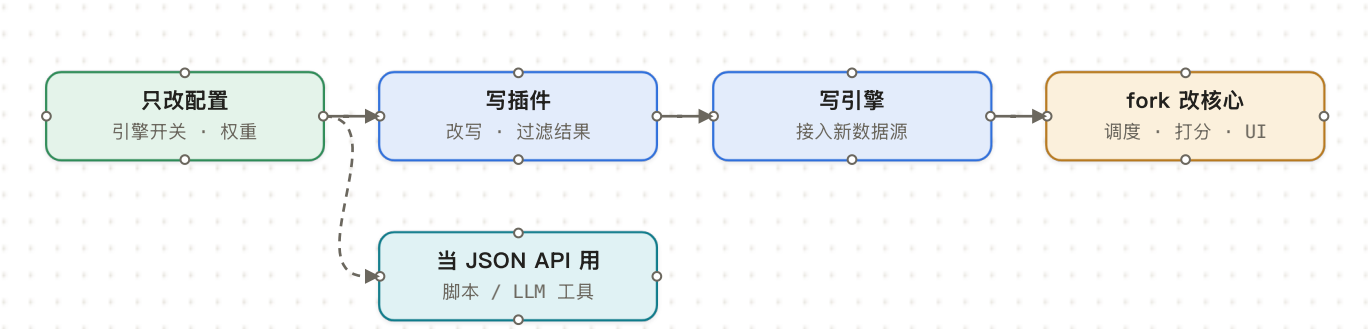
适合入手的方向

方向	为什么适合
修复坏掉的引擎	范围小、好验证；上游一改版就有人需要，改动通常只在一个文件
新增引擎	有现成的最小例子（`github.py`），或者只用 `xpath` / `json_engine` 写配置
新增结果类型或插件	有清晰的扩展点（`result_types/`、`plugins/`）
更新数据	`searxng_extra/update/` 下的脚本重新生成语言、货币、bang 数据

需要避开的：依赖某个上游私有接口、违反其服务条款的引擎；会在服务端保存用户数据的功能（和项目目标冲突）。

定制路线

按改动由小到大



越往右，跟上游合并的成本越高。前三种都可以不改核心代码：插件和引擎各是一个 Python 文件，用 `settings.yml` 启用。

路线	能做到	代价
只改配置	选引擎、调权重、改超时、走代理、改写域名（`hostnames`）、开 JSON	几乎没有，升级无痛
写插件	任意过滤、改写、补充结果	一个文件；插件 API 偶有变动
写引擎	接入公司内部搜索、知识库、某个站点	一个文件；上游改版要跟着修
fork 改核心	改调度、打分、界面	要持续合并上游；对外提供服务时按 AGPL 公开
当 JSON API 用	给脚本或大模型当搜索工具	实例要开 `json` 格式；注意上游限流

附录：源码索引

主题	位置
Flask 应用、路由、初始化	`webapp.py`：`init`、`pre_request`、`search`
默认配置	`settings.yml`
配置加载与默认值	`settings_loader.py`、`settings_defaults.py`
偏好	`preferences.py`
查询语法	`query.py`
组装 SearchQuery	`webadapter.py`、`search/models.py`
搜索调度	`search/__init__.py`
processor 与挂起	`abstract.py`、`online.py`
引擎加载	`engines/__init__.py`
引擎语言特性	`enginelib/traits.py`
网络层	`network/__init__.py`、`network.py`、`client.py`
结果容器、合并、打分	`results.py`、`calculate_score`
结果类型	`result_types/_base.py`
插件	`plugins/_core.py`
答题器	`answerers/_core.py`
外部 bang	`external_bang.py`
limiter 与机器人检测	`limiter.py`、`botdetection/`
缓存	`cache.py`
输出	`webutils.py`、`templates/simple/`
前端主题源码	`client/simple/`
部署	`container/`、`utils/searxng.sh`
数据更新脚本	`searxng_extra/update/`